Lecture 03

RISC-V: Data Transfer and Upper Immediate Instructions

Instructor: Dr. Muhammad Imran

CHIPXPRT

NCDC
NUST CHIP DESIGN CENTRE

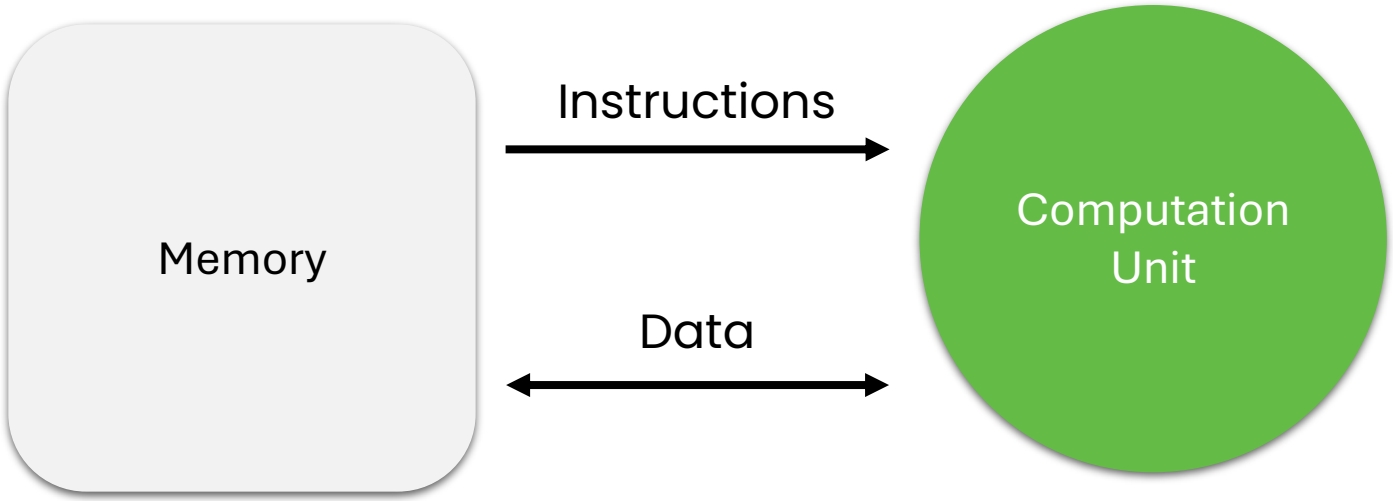
Acknowledgement

- Content from following has been used in these lectures
  - Computer Organization and Design, RISC-V 2nd Edition, Patterson and Hennessy
  - CS 61C: Great Ideas in Computer Architecture (Machine Structures), UC Berkeley

Contents

Processor Design – The Big Picture

- Mainly two components
  - **Memory** – Holds data and instructions
  - **The Computation Unit** – Performs Computation



Load and Store Instructions

Loading and Storing Data

- Data transfer instructions load operands from memory and store result to memory

- Format
  - *inst_name reg, offset(base address in register)*

Load Word

- Load word syntax:
  - lw rd, imm(rs1)
  - Means: Compute **imm+rs1**, then load the **4 bytes** (32-bit) from address **imm+rs1** into **rd**
  - Example
    - lw x7, 0(x6)
      - x7 = Memory[x6+0]
      - Registers are 32-bit in 32-bit implementation!

Load Word

- Example
  - Assume following memory contents and **x5 = 0x100**

Byte (0x)	EF	BE	AD	DE	43	53	36	31	43	20	52	49	53	42	56	00
Address (0x)	100	101	102	103	104	105	106	107	108	109	10A	10B	10C	10D	10E	10F

  - What is in x10 after executing **lw x10, 12(x5)** ?
- Solution
  - 0x100+12 = 0x10C
  - Bytes at **0x10C-0x10F** are **0x53**, **0x42**, **0x56**, **0x00**

Byte (0x)	EF	BE	AD	DE	43	53	36	31	43	20	52	49	**53**	**42**	**56**	**00**
Address (0x)	100	101	102	103	104	105	106	107	108	109	10A	10B	10C	10D	10E	10F

  - Since RISC-V is little-endian order, this is the 32-bit value **0x0056 4253**
  - So, register x10 will now store **0x0056 4253**

Exercise …

- Given following memory contents:

Byte (0x)	FF	12	6D	12	45	23	00	1E
Address (decimal)	0	1	2	3	4	5	6	7

- Let's assume **x6 = 3**
  - What will be in **x7** after executing the following instruction?
    - **lw x7, 0(x6)**
  - Solution
    - **x7 = 0x00234512**
      - **0x12** is the least-significant byte!

Store Word

- Store word syntax:
  - sw rs2, imm(rs1)
  - Means: Compute **imm+rs1**, then store the 4 bytes of **rs2** into the address **imm+rs1**
  - Example
    - **sw x5, 4(x2)**
      - Memory[x2+4] = x5

Store Word

- Example
  - Assume following memory contents

Byte (0x)	EF	BE	AD	DE	43	53	36	31	43	20	52	49	53	42	56	00
Address (0x)	100	101	102	103	104	105	106	107	108	109	10A	10B	10C	10D	10E	10F

  - If **x5** is **0x100**, **x10** is **0x1234 5678**,
  - how memory contents change after executing **sw x10, 0(x5)** ?
- Solution:
  - **0x100+0 = 0x100**
  - Since RISC-V is little-endian, the 32-bit value **0x1234 5678** gets split into bytes **0x78 0x56 0x34 0x12**
  - So, the bytes in memory **0x100**, **0x101**, **0x102**, and **0x103** get set to **0x78**, **0x56**, **x34**, and **0x12**, respectively.

Byte (0x)	**78**	**56**	**34**	**12**	43	53	36	31	43	20	52	49	53	42	56	00
Address (0x)	100	101	102	103	104	105	106	107	108	109	10A	10B	10C	10D	10E	10F

Exercise …

- Given following memory contents:

Byte (0x)	FF	12	6D	12	45	23	00	1E
Address (decimal)	0	1	2	3	4	5	6	7

- Let's assume **x6 = 2** and **x7 = 0x67854309**
  - What will be the memory contents after executing **sw x7, 0(x6)** **?**

  - Solution:

Byte (0x)	FF	12	**09**	**43**	**85**	**67**	00	1E
Address (decimal)	0	1	2	3	4	5	6	7

  - **0x67854309** is stored in little-endian order at address **x6+0 = 2**!

Loading and Storing Bytes

- In addition to word data transfers (lw, sw), RISC-V has byte data transfers:
  - lb rd, imm(rs1)
  - sb rs2, imm(rs1)
- Load and store one byte instead of a full word

- **Problem:** If registers contain 4 bytes, how do we load/store only 1 byte?

Store Byte

- Example:
  - Given memory contents:

Byte (0x)	EF	BE	AD	DE	43	53	36	31	43	20	52	49	53	42	56	00
Address (0x)	100	101	102	103	104	105	106	107	108	109	10A	10B	10C	10D	10E	10F

  - Execute **sb x10, 0(x5)** if **x5** is **0x100**, **x10** is **0x1234 5678**

- Solution:
  - For **sb**, we store the least significant byte.
  - In the above example, **0x78** is the LSB.
  - So, **x100** gets set to **0x78**

Byte (0x)	**78**	BE	AD	DE	43	53	36	31	43	20	52	49	53	42	56	00
Address (0x)	100	101	102	103	104	105	106	107	108	109	10A	10B	10C	10D	10E	10F

Load Byte

- Example:
  - Given memory contents:

Byte (0x)	EF	BE	AD	DE	43	53	36	31	43	20	52	49	53	42	56	00
Address (0x)	100	101	102	103	104	105	106	107	108	109	10A	10B	10C	10D	10E	10F

  - What happens after executing **lb x10, 0(x5)** if **x5** is **0x100** ?
- Solution:
  - For **lb**, we *extend* the numeric value to a full **32** bits.

Byte (0x)	**EF**	BE	AD	DE	43	53	36	31	43	20	52	49	53	42	56	00
Address (0x)	100	101	102	103	104	105	106	107	108	109	10A	10B	10C	10D	10E	10F

  - In the above example, we load the number. What 32-bit number has the same numeric value as **0xEF**?
  - **Answer:** It depends on your representation scheme!

Sign- and Zero-Extension

- There are two main representation schemes used: unsigned numbers, and 2's complement

- For unsigned numbers:
  - 8-bit **0xEF** = **239**
  - **239** represented in 32 bits is **0x000000EF**
  - **General rule:** Fill the top bits with 0s (Zero-Extension)

- For signed numbers:
  - 8-bit **0xEF** = **-17**
  - **-17** represented in 32 bits is **0xFFFF FFEF**
  - **General rule:** Fill the top bits with the most significant bit of the number (Sign-Extension)

Loading and Storing Bytes

- In addition to word data transfers (lw, sw), RISC-V has byte data transfers:
  - lb rd imm(rs1) → sign extend the byte
  - lbu rd imm(rs1) → zero extend the byte
  - sb rs2 imm(rs1) → store least significant byte only

- Load and store one byte instead of a full word

Exercise …

- What is in **x12** after executing following code?

```
li      x11     0x93F5
sw      x11     0(x5)
lb      x12     1(x5)
```

Register	Value
x5	0x0000 0100
x11	0x0000 0000
x12	0x0000 0000

- Memory

Byte (0x)	EF	BE	AD	DE	43	53	36	31	43	20	52	49	53	42	56	00
Address (0x)	100	101	102	103	104	105	106	107	108	109	10A	10B	10C	10D	10E	10F

Exercise …

- What is in **x12** after executing following code?

```
li      x11     0x93F5
sw      x11     0(x5)
lb      x12     1(x5)
```

Register	Value
x5	0x0000 0100
x11	**0x0000 93F5**
x12	0x0000 0000

- Memory

Byte (0x)	EF	BE	AD	DE	43	53	36	31	43	20	52	49	53	42	56	00
Address (0x)	100	101	102	103	104	105	106	107	108	109	10A	10B	10C	10D	10E	10F

Exercise …

- What is in **x12** after executing following code?

```
li      x11     0x93F5
sw      x11     0(x5)
lb      x12     1(x5)
```

Register	Value
x5	0x0000 0100
x11	**0x0000 93F5**
x12	0x0000 0000

- Memory

Byte (0x)	**F5**	**93**	**00**	**00**	43	53	36	31	43	20	52	49	53	42	56	00
Address (0x)	100	101	102	103	104	105	106	107	108	109	10A	10B	10C	10D	10E	10F

Exercise …

- What is in **x12** after executing following code?

```
li      x11     0x93F5
sw      x11     0(x5)
lb      x12     1(x5)
```

Register	Value
x5	0x0000 0100
x11	0x0000 93F5
x12	**0xFFFF FF93**

Sign-extend: Top bit of **0x93** is **1**, so fill with **1s**!

- Memory

Byte (0x)	F5	**93**	00	00	43	53	36	31	43	20	52	49	53	42	56	00
Address (0x)	100	101	102	103	104	105	106	107	108	109	10A	10B	10C	10D	10E	10F

Similar Instructions …

- **lh** – load halfword (16-bit value) → sign-extend!
- **lhu** – load halfword unsigned → zero-extend!
- **sh** – store halfword in memory!

- Note:
  - There's no **lwu** in 32-bit implementation
  - There's **lwu** in 64-bit implementation and no **ldu**!

Offset vs Element Number

- Offset is based on number of bytes!

- If each element of an array is **word-sized**,
  - then to load the element at index 3, offset = 3x4 = 12 !!

Exercise …

- Convert following C-language statement into RISC-V assembly
  - A[12] = h + A[8]
    - Assume x21 = h, x22 = base address of A
    - Use x9 as the temporary register!
    - Assume that array elements are word sized!
- Solution

```
lw x9, 32(x22)          # Temporary reg x9 gets A[8]
add x9, x21, x9         # Temporary reg x9 gets h + A[8]
sw x9, 48(x22)          # Stores h + A[8] back into A[12]
```

Another Exercise …

- Compile following C-language statement into RISC-V assembly
  - A[30] = h + A[30] + 1;
    - Assume x21 = h, x10 = base address of A
    - Use x9 as the temporary register!
    - Assume that array elements are word sized!

- Solution

```
lw x9, 120(x10)          # Temporary reg x9 gets A[30]
add x9, x21, x9          # Temporary reg x9 gets h+A[30]
addi x9, x9, 1           # Temporary reg x9 gets h+A[30]+1
sw x9, 120(x10)          # Stores h+A[30]+1 back into A[30]
```

Memory Alignment

- Data is accessed in terms of multiples of bytes from physical memory chip!
  - Example
    - Word-aligned memory may provide data at (byte) address **0**, **4**, **8** and so on!
    - What if instruction is **lw x10, 3(x0)**?
      - That's alignment violation for a word-aligned memory!
    - Solution

1. Do not allow unaligned memory access – usual approach!
   - Hardware designer expects that address will always be aligned at some boundary!
2. Allow (in implementation) unaligned memory access – not preferred!
   - Memory hardware needs to deal with it internally (taking more access time)!

Exercise – Converting C Code to RISC-V Assembly

Converting C Code to RISC-V

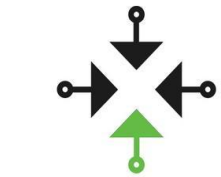
- So far, we've assumed that each variable gets stored in one register.
- What if we have more than 32 variables?

- Let's translate a program under the following restrictions:
  - Only registers x5, x6, and x7 may be modified, and only for intermediate calculations
    - We'll name them "t0", "t1", and "t2", for "temporary register 0-2“
  - x2 points to the start of a block of memory that we can use however we want
    - We'll name x2 "sp", for "stack pointer"

Converting C Code to RISC-V

```
int a = 5;

char b[] = "string";          // Array will get stored on stack

int c[10];

uint8_t d = b[3];

c[4] = a+d;

c[a] = 20;
```

We will use RISC-V instructions to define data in memory!
(not the way compiler would do it !!!)



Converting C Code to RISC-V

```
int a = 5;

char b[] = "string";

int c[10];

uint8_t d = b[3];

c[4] = a+d;

c[a] = 20;
```

Step 1:

- Assign each variable to some offset from sp.
- Exact values don't matter as long as we're consistent
  - a: 0(sp)
  - b: 4(sp)
  - c: 12(sp)
  - d: 52(sp)

Converting C Code to RISC-V

```
int a = 5;

char b[] = "string";

int c[10];

uint8_t d = b[3];

c[4] = a+d;

c[a] = 20;
```

```
a: 0(sp)
b: 4(sp)
c: 12(sp)
d: 52(sp)
```

```
li      t0      5
sw      t0      0(sp)
```

Converting C Code to RISC-V

```
int a = 5;

char b[] = "string";

int c[10];

uint8_t d = b[3];

c[4] = a+d;

c[a] = 20;
```

a: 0(sp)
b: 4(sp)
c: 12(sp)
d: 52(sp)

```
li      t0      0x73
sb      t0      4(sp)
li      t0      0x74
sb      t0      5(sp)
li      t0      0x72
sb      t0      6(sp)
li      t0      0x69
sb      t0      7(sp)
li      t0      0x6e
sb      t0      8(sp)
li      t0      0x67
sb      t0      9(sp)
sb      x0      10(sp)
```

Converting C Code to RISC-V – better approach for same!

```
int a = 5;

char b[] = "string";

int c[10];

uint8_t d = b[3];

c[4] = a+d;

c[a] = 20;
```

```
a: 0(sp)
b: 4(sp)
c: 12(sp)
d: 52(sp)
```

```
li    t0    0x69727473
sw    t0    4(sp)
li    t0    0x0000676E
sw    t0    8(sp)
```

Converting C Code to RISC-V

```
int a = 5;

char b[] = "string";

int c[10];

uint8_t d = b[3];

c[4] = a+d;

c[a] = 20;
```

a: 0(sp)

b: 4(sp)

c: 12(sp)

d: 52(sp)

Nothing …

- No code needed, since **c** is not initialized!

Converting C Code to RISC-V

```
int a = 5;

char b[] = "string";

int c[10];

uint8_t d = b[3];

c[4] = a+d;

c[a] = 20;
```

a: 0(sp)
b: 4(sp)
c: 12(sp)
d: 52(sp)

```
lb      t0      7(sp)
sb      t0      52(sp)
```

Converting C Code to RISC-V

```
int a = 5;

char b[] = "string";

int c[10];

uint8_t d = b[3];

c[4] = a+d;

c[a] = 20;
```

```
a: 0(sp)
b: 4(sp)
c: 12(sp)
d: 52(sp)
```

```
lw      t0      0(sp)
lbu     t1      52(sp)
add     t2      t0      t1
sw      t2      28(sp)
```

Converting C Code to RISC-V

```
int a = 5;

char b[] = "string";

int c[10];

uint8_t d = b[3];

c[4] = a+d;

c[a] = 20;
```

a: 0(sp)
b: 4(sp)
c: 12(sp)
d: 52(sp)

```
li      t0      20
lw      t1      0(sp)
# sw    t0      t1*4+12(sp)
slli    t1      t1      2
                # t1*=4
addi    t1      t1      12
add     t1      t1      sp
sw      t0      0(t1)
```

Converting C Code to RISC-V

```
int a = 5;

char b[] = "string";

int c[10];

uint8_t d = b[3];

c[4] = a+d;

c[a] = 20;
```

a: 0(sp)
b: 4(sp)
c: 12(sp)
d: 52(sp)

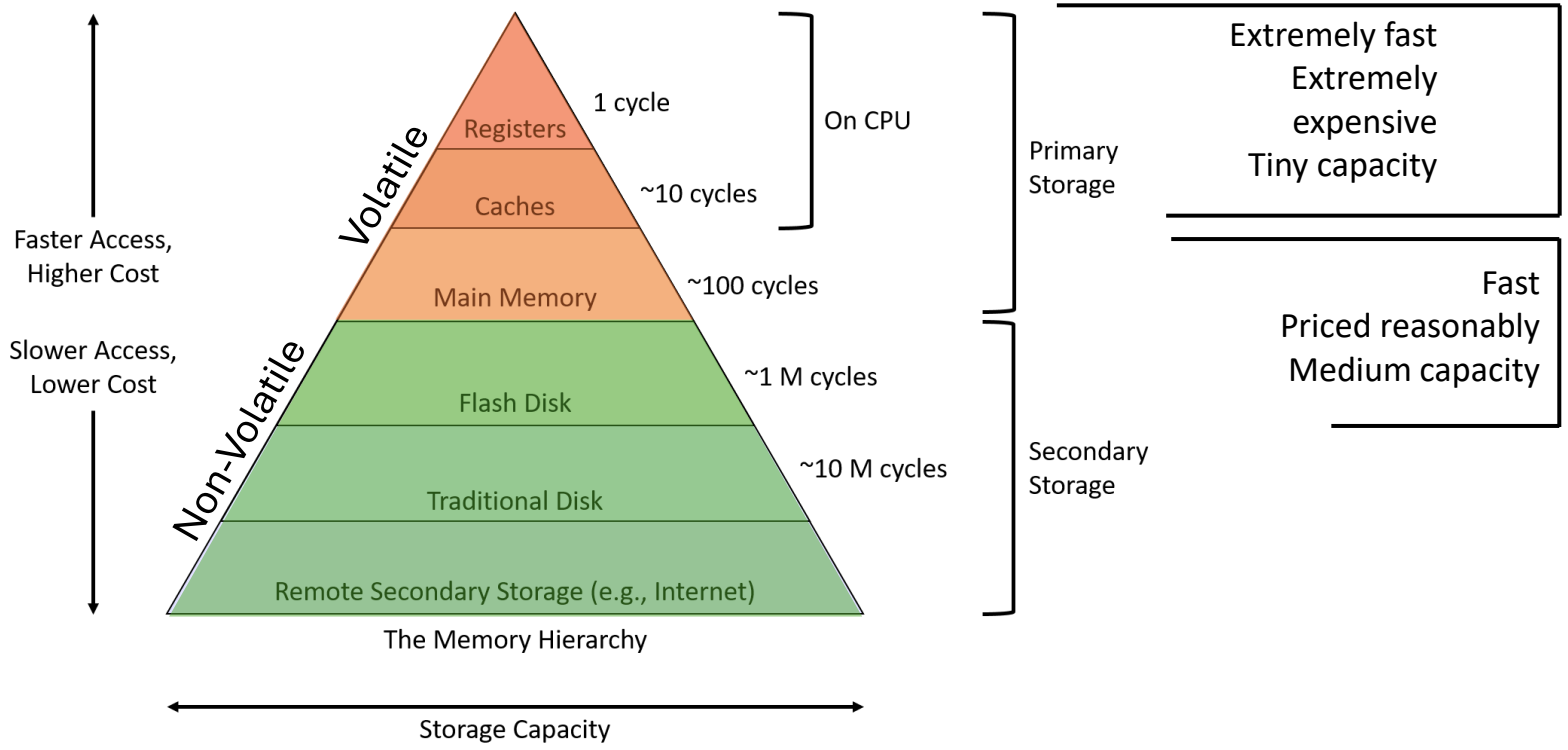
```
li      t0      5
sw      t0      0(sp)
li      t0      0x69727473
sw      t0      4(sp)
li      t0      0x0000676e
sw      t0      8(sp)
lb      t0      7(sp)
sb      t0      52(sp)
lw      t0      0(sp)
lbu     t1      52(sp)
add     t2      t0      t1
sw      t2      28(sp)
li      t0      20
lw      t1      0(sp)
slli    t1      t1      2
        # t1*=4
addi    t1      t1      12
add     t1      t1      sp
sw      t0      0(t1)
```

Why we need so many registers ?

- As the previous example showed, it's possible to write RISC-V with only a **sp** and **three** temporary registers

- Why do we have **32** registers?

RISC-V Guiding Philosophy



Credit: N. When, Chrsitian Weis, RPTU

Speed of Registers vs Memory

- Given that
  - Registers: 32 words (128 Bytes)
  - Memory (DRAM): Billions of bytes
    (2 GB to 96 GB on laptop)

- and physics dictates…
  - smaller is faster

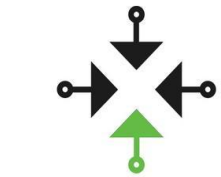
- How much faster are registers than DRAM??
  - About 50-500 times faster!
    (in terms of latency of one access - tens of ns)
  - But subsequent words come every few ns

and in conclusion …

- Memory is byte-addressable, but **lw** and **sw** access one word at a time
- A pointer (used by **lw** and **sw**) is just a memory address, we can add to it or subtract from it (using offset)
- Memory can be used for variables we can't store in registers, but 100x slower than using registers directly
  - Use loads and stores as infrequently as possible!
- New Instructions:
  - lb, lbu, lh, lhu, lw
  - sb, sh, sw

Upper Immediate Instructions

Immediate in most ALU instructions is 12-bit, how can we have larger immediate value e.g., 32-bit?



Load Upper Immediate Instruction

- **lui** instruction allows a mechanism to make larger immediate …
  - By loading 20 bits to the upper 20 bits ([31:12]) of the register!

- Example
  - Load following into 32-bit register, x19!
    - 00000000 00111101 0000**0101 00000000**

  - Solution
    - 00000000 00111101 0000 = 976 in decimal
    - 0101 00000000 = 1280 in decimal
    - lui x19, 976 # load immediate in bits {31-12} of register and make other bits 0!
    - addi x19, x19, 1280 # change lower 12 bits!

LUI – Corner case

- How would you translate **li t0 0xABCDEFFF** to instructions?
  - Initial idea:
    - lui t0 0xABCDE # upper 20 bits
      addi t0 t0 0xFFF # lower 12 bits
  - See any problem?
    - Problem: **0xFFF** isn't **4095**; it's **-1**
    - After the first line, we get **t0** = **0xABCDE000**
    - After the second line, we get **t0** = **0xABCDDFFF** instead!
  - As such, we need to be careful in this case and do lui t0 0xABCDF instead
    - **lui t0 0xABCDF** # t0 stores 0xABCDF000
    - **addi t0 t0 0xFFF** # t0 stores 0xABCDEFFF
- This ends up affecting li instructions only when the offset has its 11th bit set to 1, so it's an easy case to forget about!

AUIPC and Relative Addressing

- **auipc** similarly gets used as a way to save an arbitrary value when used with an **addi**
  - The main difference is that it **adds its result to PC!**

- Often when writing code, we want to allow multiple programs to be combined (like with libraries), but that would change the addresses of all the labels in our code
  - To avoid this issue, many instructions involving labels use relative addressing instead of absolute addressing.
    - **Absolute address:** "This label is at location **0x000000FC**". This fails if we move our code to a different place in memory
    - **Relative address:** "This label is **48** bytes after the current line of code". This still works if we move both the line of code and the label the same distance.

- As such, **auipc** often gets used with **la** (pseudo) instructions

Add Upper Immediate to PC Instruction

- **auipc** is helpful in making addresses in PC-relative addressing
  - **auipc** forms a 32-bit offset from the 20-bit immediate, filling in the lowest 12 bits with zeros, adds this offset to the **pc**, then places the result in register **rd**
    - Can be used for both control-flow transfers or data accesses!
    - Used in combination with addi, jalr or load / store …
  - See RISC-V ISA Manual for details …

Instructions covered so far …

- **Arithmetic**
  - add, sub, addi
- **Logical**
  - and, or, xor
  - andi, ori, xori
- **Shift**
  - sll, srl, sra
  - slli, srli, srai
- **Data Transfer**
  - lb, lbu, lh, lhu, lw
  - sb, sh, sw
- **Upper Immediate**
  - lui, auipc

Practice Tasks

- Using only the instructions learned so far to complete following tasks!

- Task 1
  - Convert an array of (5) byte-sized integer values, in memory, to word-sized integers while retaining the value!

- Task 2
  - Convert a string of (5) small English letters, defined in memory, to a string of capital English letters!

- Task 3
  - Reverse a string of (5) characters, defined in memory!

Thank You